Universidad del Valle de Guatemala
Facultad de Ingeniería en Ciencias de la computación e IT

Ejercicio 6: Polimorfismo vía Interfaces

Estudiante: Fabricio Roberto Xiloj Molina
Carné: 262140
Fecha: 8/10/2026

1. Situación

Una tienda en línea necesita comunicarse con diferentes servicios externos, como pasarelas de pago, billeteras digitales, empresas de envíos y certificadores FEL.

Cada servicio funciona diferente, pero el programa debe poder manejarlos desde un mismo lugar.

También se necesita que en el futuro sea fácil agregar nuevos servicios sin cambiar todo el programa.

2. Objetivo

Crear un programa sencillo:

- Ver todas las integraciones.
- Buscar una integración por su código.
- Ordenar las integraciones por su latencia.
- Realizar cobros.
- Realizar reintegros.
- Crear documentos.
- Registrar auditorías.
- Ignorar operaciones que un servicio no puede realizar.

3. Solución propuesta

Se creó una clase general llamada ApiExterna.

Esta clase guarda los datos que todas las integraciones tienen en común:

- Código.
- Nombre.
- Latencia promedio.
- Estado.

Después se crearon cuatro clases:

- PasarelaPago
- BilleteraDigital
- ProveedorLogistico
- CertificadorFEL

Estas clases heredan de ApiExterna.

4. Interfaces utilizadas

Transaccional

Permite realizar operaciones de dinero:

- Cobrar.
- Reintegrar.

La utilizan PasarelaPago y BilleteraDigital.

Documentable

Permite crear documentos.

La utilizan ProveedorLogistico y CertificadorFEL.

Auditable

Permite registrar las operaciones realizadas.

La utilizan todas las integraciones.

5. Lista polimórfica

El programa utiliza una sola lista para guardar todas las integraciones.

La lista es de tipo Integracion, por lo que puede guardar objetos de diferentes clases.

Al iniciar el programa se agregan 10 integraciones:

1. VisaPay.
2. MasterCard.
3. PayFast.
4. Maya Wallet.
5. WalletGo.
6. Envío Rápido.
7. Carga Ya.
8. Última Milla.
9. FEL Guatemala.
10. Factura Now.

Esto permite trabajar con todos los servicios desde una sola lista.

6. Herencia y polimorfismo

La clase ApiExterna tiene la información común de todas las integraciones.

Las clases específicas heredan de ella.

El polimorfismo permite que el programa trate todos los servicios como una integración general, aunque cada uno tenga funciones diferentes.

Por ejemplo, una pasarela de pago puede cobrar, mientras que un proveedor logístico puede crear una guía de envío.

7. Caso del proveedor logístico

Un proveedor logístico no puede realizar cobros ni reintegros.

Si el usuario intenta hacer un reintegro usando un proveedor logístico, el programa muestra un mensaje e ignora la solicitud.

El programa no se cierra ni muestra un error.

8. Organización del programa

El programa está dividido en tres partes:

Modelo

Contiene las clases y las interfaces.

Vista

Contiene el menú de consola y muestra los mensajes al usuario.

Controlador

Conecta la vista con el modelo y se encarga de ejecutar las operaciones.

9. Sobrecarga

Se utilizó sobrecarga en los constructores de ApiExterna.

También se utilizó sobrecarga en el método agregar del catálogo.

10. Menú del programa

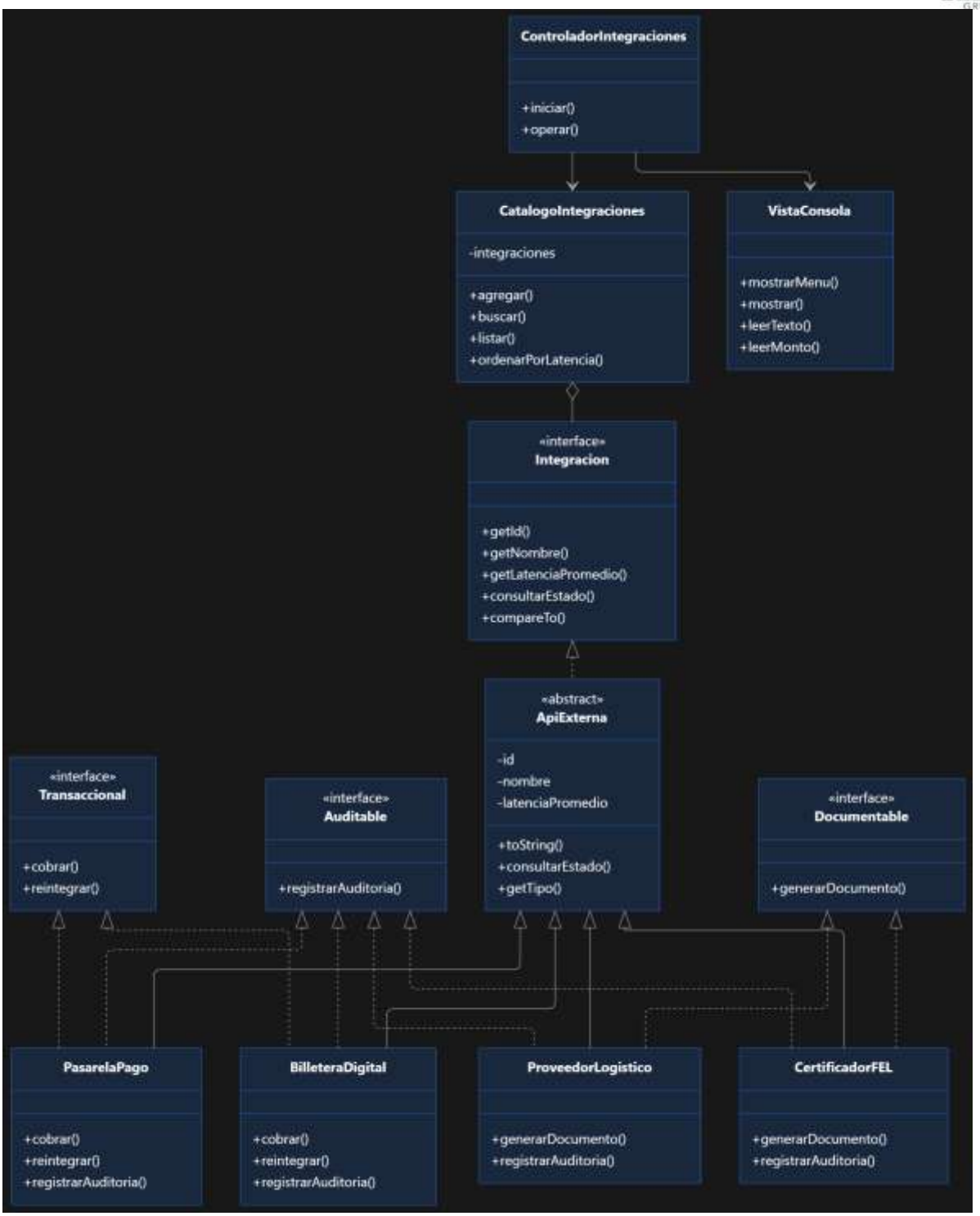
El menú principal permite:

1. Listar integraciones.
2. Buscar una integración.
3. Ordenar por latencia.
4. Ejecutar una operación.
5. Salir.

Las operaciones disponibles son:

1. Cobrar.
2. Reintegrar.
3. Generar documento.
4. Registrar auditoría.

11. Diagrama UML


ControladorIntegraciones
+iniciar()
+operar()
CatalogoIntegraciones
-integraciones
+agregar()
+buscar()
+listar()
+ordenarPorLatencia()
VistaConsola
+mostrarMenu()
+mostrar()
+leerTexto()
+leerMonto()
«interface»
Integracion
+getId()
+getNombre()
+getLatenciaPromedio()
+consultarEstado()
+compareTo()
«abstract»
ApiExterna
-id
-nombre
-latenciaPromedio
+toString()
+consultarEstado()
+getTipo()
«interface»
Transaccional
+cobrar()
+reintegrar()
«interface»
Auditable
+registrarAuditoria()
«interface»
Documentable
+generarDocumento()
PasarelaPago
+cobrar()
+reintegrar()
+registrarAuditoria()
BilleteraDigital
+cobrar()
+reintegrar()
+registrarAuditoria()
ProveedorLogistico
+generarDocumento()
+registrarAuditoria()
CertificadorFEL
+generarDocumento()
+registrarAuditoria()

c

12. Posibles mejoras

En el futuro se podría agregar una nueva empresa de envíos o una nueva pasarela de pago.

Solamente sería necesario crear una nueva clase, heredar de ApiExterna, implementar las interfaces necesarias y agregarla al catálogo.

13. Conclusión

El programa permite manejar diferentes servicios externos usando una sola lista y varias interfaces.

Se utilizó herencia para compartir información y polimorfismo para trabajar con diferentes tipos de integraciones.

También se utilizó una separación sencilla entre modelo, vista y controlador.